

ML SUMMARY FOR EXAM

LOGISTIC REGRESSION

How the model can be extended and applied to address score calibration issues

In the generative model (Bayes), decisions are computed using the posterior log-odds ratio: $\log \frac{P(C=1|x)}{P(C=0|x)} \geq -\log \frac{C_{fn}}{C_{fp}}$. If: $C_{fn} = C_{fp}$ then: $t = 0$. In Logistic Regression, however, we use the empirical prior of the training set, while Bayes decision theory uses the application prior.

Using Bayes rule: $\log \frac{P(C=1|x)}{P(C=0|x)} = \log \frac{f(x|C=1)P(C=1)}{f(x|C=0)P(C=0)}$ which can be rewritten as:

$$s_{post}(x) = s_{llr}(x) + \log \frac{\pi}{1-\pi}$$

where:

- $s_{llr}(x)$ is the log-likelihood ratio,
- π is the class prior.

For Logistic Regression: $s_{post}(x) = w^T x + b$. Therefore: $s_{post}(x) = s_{llr}(x) + \log \frac{\pi}{1-\pi}$

and thus: $s_{llr}(x) = s_{post}(x) - \log \frac{\pi}{1-\pi}$. Since Logistic Regression is trained using the empirical training prior: $\pi_{emp}^{tr} = \frac{n_T}{n}$ and $1 - \pi_{emp}^{tr} = \frac{n_F}{n}$ we obtain:

$$\log \frac{\pi_{emp}^{tr}}{1 - \pi_{emp}^{tr}} = \log \frac{n_T}{n_F}$$

Hence, to recover a score behaving like a log-likelihood ratio, we subtract from the score the empirical prior log-odds of the training set: $s_{llr} = w^T x + b - \log \frac{n_T}{n_F}$

equivalently: $s_{llr} = w^T x + b - \log \frac{\pi_{emp}^{tr}}{1 - \pi_{emp}^{tr}}$

We can then use s_{llr} as a log-likelihood ratio.

For a given application $(\pi_T, 1, 1)$, we can compute decisions by comparing:

$$s_{llr} \geq -\log \frac{\pi_T}{1 - \pi_T}$$

This becomes the new Bayes-optimal threshold.

However, there is still an important limitation.

While this allows effective decisions for the model (w, b) , the orientation of w is still optimized for the empirical prior of the training set. Changing the threshold only changes the operating point, but only among hyperplanes orthogonal to w . If we know the application prior π_T before training, it may be preferable to directly optimize the separation rule for the target application prior. This leads to prior-weighted Logistic Regression. The idea is to weight the loss differently for the two classes according to the target prior:

$$R(w, b) = \frac{\lambda}{2} \|w\|^2 + \pi_T \frac{1}{n_T} \sum_{i: y_i=1} \log(1 + e^{-z_i}) + (1 - \pi_T) \frac{1}{n_F} \sum_{i: y_i=0} \log(1 + e^{z_i})$$

where:

- positive and negative samples contribute differently,
- the classifier is optimized directly for the target application prior,
- the resulting separating hyperplane is better suited for the target operating condition.

SVM VS LR (risk minimization approaches)

Both LR and SVM learn a decision function by minimizing a cost over the training data, so they can be interpreted as RISK MINIMIZATION METHODS.

In LOGISTIC REGRESSION, the classifier assumes a linear score: $s(x) = w^T x + b$

This score is converted into a posterior probability through a SIGMOID FUNCTION:

$$P(C = 1 | x) = \sigma(s(x)) = \frac{1}{1 + e^{-s(x)}} = \frac{1}{1 + e^{-(w^T x + b)}}$$

LR is a PROBABILISTIC DISCRIMINATIVE CLASSIFIER: It directly model the posterior probability of the class given the input data.

w, b are estimated by MAXIMIZING the conditional likelihood of the training labels, so by MINIMIZING the binary cross entropy, the negative log_likelihood. For labels $z_i \in \{-1, +1\}$, the objective can be written as:

$$J(w, b) = \sum_{i=1}^N \log(1 + e^{-z_i(w^T x_i + b)}) \rightarrow \text{LOGISTIC LOSS (empirical risk minimization} \rightarrow$$

the risk is the sum of losses over the observed samples).

With regularization it becomes:

$$J(w, b) = \frac{\lambda}{2} \|w\|^2 + \sum_{i=1}^N \log(1 + e^{-z_i(w^T x_i + b)})$$

The first term controls model complexity, while the second measures the number of misclassified points. The logistic loss is smooth and never exactly zero: even correctly classified samples still contribute a small loss.

In **SVM**, the goal is not to estimate probabilities. Instead, SVM searches for a hyperplane that separates the classes with the **largest margin**. For linearly separable data, the hard-margin SVM solves

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

subject to

$$z_i(w^T x_i + b) \geq 1$$

This means that every training sample must be correctly classified and must lie outside the margin. Minimizing $\|w\|^2$ is equivalent to maximizing the margin.

or non-separable data, SVM uses the **hinge loss**:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - z_i(w^T x_i + b))$$

The hinge loss is

$$l_{\text{SVM}}(s) = \max(0, 1 - s)$$

while the Logistic Regression loss is

$$l_{\text{LR}}(s) = \log(1 + e^{-s})$$

The key difference is that Logistic Regression penalizes all samples, while SVM only penalizes samples that are misclassified or inside the margin. If $z_i(w^T x_i + b) > 1$, the hinge loss is zero. Therefore, only some training samples influence the final SVM solution: these are the **support vectors**.

So, the objective functions can be compared as follows:

$$\begin{aligned} \text{Logistic Regression: } & \frac{1}{2} \|w\|^2 + C \sum_i \log(1 + e^{-z_i s_i}) \\ \text{SVM: } & \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - z_i s_i) \end{aligned}$$

where $s_i = w^T x_i + b$.

The main conceptual difference is that **Logistic Regression minimizes a probabilistic loss**, derived from maximum likelihood and cross-entropy, while **SVM minimizes a margin-based loss**, derived from the idea of maximizing the distance between the decision boundary and the closest training samples. Also, Logistic Regression outputs posterior probabilities, whereas SVM outputs a score that is useful for classification but is not directly interpretable as a posterior probability.

For **non-linear decision functions**, both models can be extended by mapping the original input x into a transformed feature space:

$$\phi(x)$$

Then the classifier becomes linear in the transformed space:

$$s(x) = w^T \phi(x) + b$$

but non-linear in the original input space.

For **Logistic Regression**, this is usually done explicitly. For example, one can define polynomial features such as

$$\phi(x) = \begin{bmatrix} \text{vec}(xx^T) \\ x \end{bmatrix}$$

and then train Logistic Regression on $\phi(x)$ instead of x . The slides explain that this gives a linear separation surface in the expanded feature space, corresponding to a quadratic separation surface in the original space.

Logistic Regression can use any explicit feature expansion: polynomial features, quadratic terms, interaction terms, or manually designed basis functions. The drawback is that the dimensionality of the expanded space can grow very quickly, especially for high-degree polynomial expansions.

In SVM we have the KERNEL TRICK:

Instead of explicitly computing $\phi(x)$, we define a kernel function

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

and replace every dot product with this kernel. The resulting classifier is

$$s(x) = \sum_{i: \alpha_i > 0} \alpha_i z_i K(x_i, x) + b$$

Only the support vectors, where $\alpha_i > 0$, contribute to the decision function

The important point is that SVM can work in a high-dimensional or even infinite-dimensional feature space without explicitly constructing that space.

In conclusion: both Logistic Regression and SVM minimize a regularized empirical risk. Logistic Regression minimizes **logistic loss**, has a probabilistic interpretation, and outputs posterior probabilities. SVM minimizes **hinge loss**, has a geometric margin interpretation, and focuses on support vectors. For non-linear classification, Logistic Regression usually needs an explicit feature transformation, while SVM can use kernels to obtain non-linear decision boundaries implicitly and efficiently.

PCA VS LDA

Dimensionality reduction compute a mapping from n-dimensional feature space to a m-dimensional space where $m \ll n$. It compress information, remove unwanted noise and simplify data visualization and classification.

1. Goals and formulation

Principal Component Analysis (PCA) is an **unsupervised** dimensionality reduction method. It does **not use class labels**. Its goal is to find a lower-dimensional subspace that preserves as much information as possible, usually by preserving the directions of **maximum variance**. Given centered data:

$$X = \{x_1, \dots, x_K\}, x_i \in \mathbb{R}^n$$

we look for a projection matrix:

$$P \in \mathbb{R}^{n \times m}, P^T P = I$$

where the columns of P are orthonormal basis vectors of the PCA subspace.

The projected sample is:

$$y_i = P^T x_i$$

and the reconstruction in the original space is:

$$\hat{x}_i = P y_i = P P^T x_i$$

So PCA searches for a subspace that reconstructs the data with minimum error.

So it wants to minimize the AVERAGE RECONSTRUCTION ERROR (K is the number of samples):

$$L(P) = \frac{1}{K} \sum_{i=1}^K \|x_i - \hat{x}_i\|^2$$

Since:

$$\hat{x}_i = PP^T x_i$$

we have:

$$L(P) = \frac{1}{K} \sum_{i=1}^K \|x_i - PP^T x_i\|^2 \text{ (training objective).}$$

So the GOAL IS:

$$P^* = \arg \min_P \frac{1}{K} \sum_{i=1}^K \|x_i - PP^T x_i\|^2$$

$$\|x_i - PP^T x_i\|^2 = (x_i - PP^T x_i)^T (x_i - PP^T x_i)$$

$$(a - b)^T (a - b) = a^T a - a^T b - b^T a + b^T b$$

$$b = PP^T x_i$$

$$a = x_i$$

So we obtain: $x_i^T x_i - x_i^T PP^T x_i - (PP^T x_i)^T x_i + (PP^T x_i)^T (PP^T x_i)$

Since: $(AB)^T = B^T A^T$ $(PP^T x_i)^T = x_i^T (PP^T)^T$, and $(PP^T)^T = PP^T$

So: $(PP^T x_i)^T x_i = x_i^T PP^T x_i$

$$-x_i^T PP^T x_i - x_i^T PP^T x_i = -2x_i^T PP^T x_i$$

Then we have: $(PP^T x_i)^T (PP^T x_i)$ so: $x_i^T (PP^T)^T PP^T x_i$, $(PP^T)^T = PP^T$

we have: $x_i^T PP^T PP^T x_i$

Finally we obtain:

$$\|x_i - PP^T x_i\|^2 = x_i^T x_i - 2x_i^T PP^T x_i + x_i^T PP^T PP^T x_i$$

Then we use this constraint $\rightarrow P^T P = I$ (columns of P are orthonormal)

We can group terms like this: $PP^T PP^T = P(P^T P)P^T$.

Since: $P^T P = I \rightarrow P(P^T P)P^T = PIP^T \rightarrow PIP^T = PP^T \rightarrow PP^T PP^T = PP^T$

Finally: $x_i^T PP^T PP^T x_i = x_i^T PP^T x_i$

We obtain: $\|x_i - PP^T x_i\|^2 = x_i^T x_i - 2x_i^T PP^T x_i + x_i^T PP^T x_i$

$$-2x_i^T PP^T x_i + x_i^T PP^T x_i = -x_i^T PP^T x_i$$

Quindi:

$$\|x_i - PP^T x_i\|^2 = x_i^T x_i - x_i^T PP^T x_i$$

For all the samples: $L(P) = \frac{1}{K} \sum_{i=1}^K \|x_i - PP^T x_i\|^2$

quindi: $L(P) = \frac{1}{K} \sum_{i=1}^K (x_i^T x_i - x_i^T PP^T x_i)$

Now we want to use the TRACE: We want to rewrite: $x_i^T PP^T x_i$ with the trace. If an expression is scalar, it is equal to its trace $\rightarrow a = \text{Tr}(a)$. So: $x_i^T PP^T x_i = \text{Tr}(x_i^T PP^T x_i)$. Now we use this property: $\text{Tr}(ABC) = \text{Tr}(BCA) = \text{Tr}(CAB)$

Starting from: $\text{Tr}(x_i^T PP^T x_i)$. Thanks to the property: $\text{Tr}(x_i^T PP^T x_i) = \text{Tr}(P^T x_i x_i^T P)$, so
:

Substituting: $L(P) = \frac{1}{K} \sum_{i=1}^K x_i^T x_i - \frac{1}{K} \sum_{i=1}^K \text{Tr}(P^T x_i x_i^T P)$. Trace is linear so: :

$$\sum_i \text{Tr}(A_i) = \text{Tr}(\sum_i A_i) \rightarrow \frac{1}{K} \sum_{i=1}^K \text{Tr}(P^T x_i x_i^T P) = \text{Tr}\left(P^T \left[\frac{1}{K} \sum_{i=1}^K x_i x_i^T\right] P\right)$$

At the end we obtain: $L(P) = \frac{1}{K} \sum_{i=1}^K x_i^T x_i - \text{Tr}\left(P^T \left[\frac{1}{K} \sum_{i=1}^K x_i x_i^T\right] P\right)$

This matrix is the empirical covariance matrix: $\frac{1}{K} \sum_{i=1}^K x_i x_i^T$, so we obtain:

$$L(P) = \frac{1}{K} \sum_{i=1}^K x_i^T x_i - \text{Tr}(P^T C P)$$

The first term doesn't depend on P , so minimizing $L(P)$ is equal to maximize $\text{Tr}(P^T C P)$

Therefore minimizing reconstruction error is equivalent to maximizing: $\text{Tr}(P^T C P)$

where: $C = \frac{1}{K} \sum_{i=1}^K x_i x_i^T$. If data are not centered, we first compute: $\bar{x} = \frac{1}{K} \sum_{i=1}^K x_i$

$$z_i = x_i - \bar{x}$$

and use the covariance matrix:

$$C = \frac{1}{K} \sum_{i=1}^K z_i z_i^T = \frac{1}{K} \sum_{i=1}^K (x_i - \bar{x})(x_i - \bar{x})^T$$

So the PCA optimal solution is $P^* = [u_1, \dots, u_m]$, where $u_1, \dots, u_m$ are the eigenvectors of C corresponding to the m largest eigenvalues.

6. Eigendecomposition in PCA

We start from the covariance matrix: $C = \frac{1}{K} \sum_{i=1}^K x_i x_i^T$. Assuming the data are centered, so: $\bar{x} = 0$ (mean). The covariance matrix tells us how the data vary in the

original feature space. An eigenvector u of C satisfies this condition: $Cu = \lambda u$. This means that applying C to u does not change the direction of u . It only scales it by λ . So: u = direction, λ = variance of the data along that direction.

If λ is large, the data are very spread along u . If λ is small, the data are compressed along u .

Since C is symmetric: $C = C^T$, it admits the eigendecomposition: $C = U\Sigma U^T$ where: $U = [u_1, u_2, \dots, u_n]$ is the matrix containing the eigenvectors as columns, and:

$$\Sigma = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix}$$

contains the eigenvalues. The eigenvalues are ordered as: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Therefore: u_1 is the direction with maximum variance, u_2 is the direction with the second largest variance, and so on. Because C is symmetric, its eigenvectors are orthonormal. This means: $u_i^T u_j = 0$ if $i \neq j$ and $u_i^T u_i = 1$. Therefore: $U^T U = I$ and also: $U U^T = I$. So U is an orthogonal matrix. To express a sample x in the PCA coordinate system, we project it onto the eigenvectors: $y = U^T x$.

Explicitly:

$$y = \begin{bmatrix} u_1^T x \\ u_2^T x \\ \vdots \\ u_n^T x \end{bmatrix}$$

So each coordinate of y tells us how much x lies along one principal direction.

Now we want to see what happens to the covariance matrix after transforming the data. The transformed data are: $y = U^T x$. The covariance of y is: $C' = \mathbb{E}[yy^T]$. Substitute: $y = U^T x$. Then: $C' = \mathbb{E}[(U^T x)(U^T x)^T]$. Now use: $(AB)^T = B^T A^T$, so: $(U^T x)^T = x^T U$. Therefore: $C' = \mathbb{E}[U^T x x^T U]$. Since U is constant with respect to the expectation: $C' = U^T \mathbb{E}[x x^T] U$, but $\mathbb{E}[x x^T] = C$. Thus: $C' = U^T C U$.

We know: $C = U \Sigma U^T$, substitute this inside: $C' = U^T C U$, so: $C' = U^T (U \Sigma U^T) U$

Now use associativity: $C' = (U^T U) \Sigma (U^T U)$. Since: $U^T U = I$, we get: $C' = I \Sigma I$ and therefore: $C' = \Sigma$. This matrix is diagonal

$$C' = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix}$$

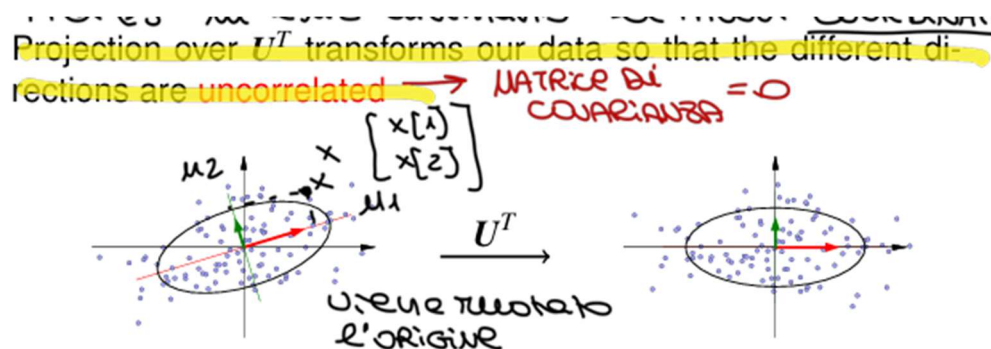
So the transformed features are uncorrelated and the covariance between different PCA components is zero; the variance of the first component is λ_1 ; the variance of the second component is λ_2 ; and so on.

PCA can be interpreted geometrically as a **rotation** of the coordinate system toward the natural axes of the data distribution. If the original features are correlated, the data cloud forms a tilted (ruotata) ellipse in the feature space. The covariance matrix: C describes both the spread and the orientation of this ellipse. By computing the eigendecomposition: $C = U\Sigma U^T$ we obtain: the eigenvectors (columns of U), which define the directions of the ellipse axes; the eigenvalues (diagonal elements of Σ), which measure the variance along those directions.

The first eigenvector corresponds to the direction with maximum variance (major axis of the ellipse), while the following eigenvectors correspond to directions with progressively smaller variance.

PCA projects the data onto the eigenvectors: $y = U^T x$. This operation changes the coordinate system from the original axes to the principal axes of the ellipse. In other words, PCA rotates the axes so that they become aligned with the data distribution.

After this transformation, the covariance matrix becomes: $C' = U^T C U = \Sigma$. Since Σ is diagonal, all off-diagonal covariance terms become zero. This means that the new variables are uncorrelated, and each principal component contains an independent amount of variance equal to its corresponding eigenvalue. Therefore, PCA finds the orthogonal directions that best describe the data variability and uses them as the new coordinate system.



4. Characteristics of PCA components and LDA directions

PCA components

PCA principal components are:

- orthogonal;
- ordered by decreasing variance;

- eigenvectors of the covariance matrix;
- independent of class labels;
- optimal for reconstruction among linear projections.

PCA preserves the largest variance directions, but these directions are not necessarily useful for classification.

5. Use in classification tasks

PCA for classification

PCA can be used as a preprocessing step before classification.

The procedure is:

1. Compute the sample mean
2. Center the data: $x_i - \bar{x}$
3. Compute the covariance matrix.
4. Compute its eigendecomposition.
5. Keep the first m eigenvectors (through cross validation using the validation set).

We choose m as the lowest number for which the sum of the first m eigenvalues divided by the sum of all eigenvalues is larger than t

$$\min_m m \quad \text{s.t.} \quad \frac{\sum_{i=1}^m \sigma_i}{\sum_{i=1}^n \sigma_i} > t$$

% di VARIANZA
[90%, 99,9%]

where σ_i is the i -th largest eigenvalue (diagonal element of Σ)

95% = t)

6. Project training and test samples: $y_i = P^T (x_i - \bar{x})$. Then a classifier can be trained in the reduced space.

For example, using a nearest class mean classifier (Euclidean classifier as for the MNIST dataset): for a test sample y_t we predict its label as the label of the class whose mean is closest to the test sample itself. $\hat{c} = \arg \min_c \|y_t - \mu_c^{(y)}\|^2$. For the Euclidean classifier on MNIST for example, PCA is not helpful in removing unwanted variability.

PCA reduces dimensionality and can reduce overfitting, but it does not guarantee better class separation because it ignores labels.

LDA GOALS

Linear Discriminant Analysis (LDA) is a **supervised** dimensionality reduction method. It uses class labels. Its goal is to find directions that maximize class separation. For each class c , define the class mean: $\mu_c = \frac{1}{n_c} \sum_{i=1}^{n_c} x_{c,i}$ and the global

mean: $\mu = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} x_{c,i}$.

LDA defines two scatter matrices:

$$S_B = \frac{1}{N} \sum_{c=1}^C n_c (\mu_c - \mu)(\mu_c - \mu)^T$$

$$S_W = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} (x_{c,i} - \mu_c)(x_{c,i} - \mu_c)^T$$

where:

- $S_B \rightarrow$ measures between-class variability. It can be interpreted as a covariance matrix for the class means, where each class is weighted by the corresponding sample size n_c . It measures how much the means of the classes are distant. If the classes are very well separated, S_B is larger.
- $S_W \rightarrow$ measures within-class variability. It can be interpreted as a weighted average of the covariance matrix of each class. It measures how much points are spread inside each class. If the classes are compact, S_W is small.

So the total variability of the dataset is:

$$S_B + S_W = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} (x_{c,i} - \mu)(x_{c,i} - \mu)^T$$

The global mean projected onto w is: $m = w^T \mu$. The mean of class c projected onto w is: $m_c = w^T \mu_c$. So projection transforms: $\mu \mapsto w^T \mu$ and: $\mu_c \mapsto w^T \mu_c$.

The between-class variance after projection is:

$$s_B = \frac{1}{N} \sum_{c=1}^C n_c (m_c - m)^2$$

Substitute:

$$m_c = w^T \mu_c, m = w^T \mu$$

Then:

$$s_B = \frac{1}{N} \sum_{c=1}^C n_c (w^T \mu_c - w^T \mu)^2$$

Factor w^T :

$$w^T \mu_c - w^T \mu = w^T (\mu_c - \mu)$$

So:

$$s_B = \frac{1}{N} \sum_{c=1}^C n_c [w^T (\mu_c - \mu)]^2$$

Now rewrite the square as a product:

$$[w^T (\mu_c - \mu)]^2 = w^T (\mu_c - \mu) (\mu_c - \mu)^T w$$

Therefore:

$$s_B = \frac{1}{N} \sum_{c=1}^C n_c w^T (\mu_c - \mu) (\mu_c - \mu)^T w$$

Since w does not depend on c , we can move it outside the summation:

$$s_B = w^T \left[\frac{1}{N} \sum_{c=1}^C n_c (\mu_c - \mu) (\mu_c - \mu)^T \right] w$$

The term inside brackets is exactly S_B . Hence:

$$s_B = w^T S_B w$$

The within-class variance after projection is: $s_W = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} (y_{c,i} - m_c)^2$

Substitute: $y_{c,i} = w^T x_{c,i}$, $m_c = w^T \mu_c$

Then:

$$s_W = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} (w^T x_{c,i} - w^T \mu_c)^2$$

Factor w^T : $w^T x_{c,i} - w^T \mu_c = w^T (x_{c,i} - \mu_c)$

So:

$$s_W = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} [w^T (x_{c,i} - \mu_c)]^2$$

Rewrite the square: $[w^T (x_{c,i} - \mu_c)]^2 = w^T (x_{c,i} - \mu_c) (x_{c,i} - \mu_c)^T w$

Therefore: $s_W = \frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} w^T (x_{c,i} - \mu_c) (x_{c,i} - \mu_c)^T w$

Move w^T and w outside: $s_W = w^T \left[\frac{1}{N} \sum_{c=1}^C \sum_{i=1}^{n_c} (x_{c,i} - \mu_c) (x_{c,i} - \mu_c)^T \right] w$

The term inside brackets is exactly S_W . Hence: $s_W = w^T S_W w$

LDA objective

LDA searches for a direction w such that, after projection: $y = w^T x$ the classes are well separated. We want larger distance between the class means and low variance inside each class. The projected between-class variance is: $s_B = w^T S_B w$.

The projected within-class variance is: $s_W = w^T S_W w$. LDA maximizes the ratio:

$$\mathcal{L}(w) = \frac{s_B}{s_W}$$

So, thanks to Fisher's criterion $\rightarrow J(w) = \frac{w^T S_B w}{w^T S_W w}$. We want to find a direction with large separation between classes and small spread inside each class. Which direction gives the best class separation? Usually we use both LDA and PCA. PCA to reduce dimensionality and compress, LDA to classify. Our goal is to find the direction u where we have between class variance larger than within class variance.

The objective is:

$$w^* = \arg \max_w \frac{w^T S_B w}{w^T S_W w}$$

The criterion does not depend on the scale of w .

$$\mathcal{L}(\alpha w) = \frac{(\alpha w)^T S_B (\alpha w)}{(\alpha w)^T S_W (\alpha w)}$$

If we replace w with αw , then:

$$\text{Expand numerator and denominator: } \mathcal{L}(\alpha w) = \frac{\alpha^2 w^T S_B w}{\alpha^2 w^T S_W w}$$

$$\text{Cancel } \alpha^2: \mathcal{L}(\alpha w) = \frac{w^T S_B w}{w^T S_W w} = \mathcal{L}(w)$$

So only the direction of w matters, not its length.

To MAXIMIZE the OBJ function we need to set the gradient to zero:

Let:

$$A(w) = w^T S_B w \text{ and } B(w) = w^T S_W w. \text{ Then: } \mathcal{L}(w) = \frac{A(w)}{B(w)}$$

Using the quotient rule of the derivatives:

$$\begin{aligned} \nabla_w \mathcal{L}(w) &= \frac{B(w) \nabla_w A(w) - A(w) \nabla_w B(w)}{B(w)^2} \\ \nabla_w (w^T A w) &= (A + A^T) w = 2Aw \text{ if } A \text{ is symmetric} \end{aligned}$$

Since S_B and S_W are symmetric:

$$\begin{aligned} \nabla_w A(w) &= 2S_B w \\ \nabla_w B(w) &= 2S_W w \end{aligned}$$

Therefore:

$$\nabla_w \mathcal{L}(w) = \frac{(w^T S_W w) 2S_B w - (w^T S_B w) 2S_W w}{(w^T S_W w)^2}$$

So:

$$\nabla_w \mathcal{L}(w) = 2 \frac{S_B w}{w^T S_W w} - 2 \frac{(w^T S_B w) S_W w}{(w^T S_W w)^2}$$

At the optimum: $\nabla_w \mathcal{L}(w) = 0$

Therefore:

$$2 \frac{S_B w}{w^T S_W w} - 2 \frac{(w^T S_B w) S_W w}{(w^T S_W w)^2} = 0$$

$$\text{Divide by 2: } \frac{S_B w}{w^T S_W w} - \frac{(w^T S_B w) S_W w}{(w^T S_W w)^2} = 0$$

Multiply by: $(w^T S_W w)^2$

Then: $(w^T S_W w) S_B w - (w^T S_B w) S_W w = 0$

So: $(w^T S_W w) S_B w = (w^T S_B w) S_W w$

Divide both sides by: $w^T S_W w$

We get: $S_B w = \frac{w^T S_B w}{w^T S_W w} S_W w$

Define: $\lambda(w) = \frac{w^T S_B w}{w^T S_W w}$

Then: $S_B w = \lambda S_W w \rightarrow$ This is a generalized eigenvalue problem.

If S_W is invertible, multiply by S_W^{-1} : $S_W^{-1} S_B w = \lambda w$

This has the standard eigenvalue form: $A w = \lambda w$

where: $A = S_W^{-1} S_B$

The eigenvalue is:

$$\lambda(w) = \frac{w^T S_B w}{w^T S_W w}$$

But this is exactly: $\mathcal{L}(w)$. Therefore: $\lambda(w) = \mathcal{L}(w)$

So the eigenvalue tells us the value of the Fisher criterion for that direction. Thus the LDA directions are the eigenvectors of: $S_W^{-1} S_B$ corresponding to the largest eigenvalues.

LDA directions

LDA discriminant directions are:

- supervised;
- chosen using class labels;
- eigenvectors of $S_W^{-1} S_B$;
- ordered by discriminative power;
- not necessarily orthogonal.

LDA directions maximize separation between classes while minimizing the spread inside each class. For a binary problem, the LDA direction is: $w \propto S_W^{-1} (\mu_2 - \mu_1)$

This means that LDA does not simply connect the two class means. It also considers the shape and spread of the classes through S_W^{-1} .

The maximum number of LDA directions is: $C - 1$ because the rank of S_B is at most $C - 1$.

LDA for classification

LDA can be used in two main ways:

1. **Directly as a binary linear classifier**
2. **As supervised dimensionality reduction before another classifier**

The slides first present LDA as a method that finds a direction w where classes are better separated. A sample x is projected onto that direction by: $y = w^T x$. The objective is to find w such that the projected classes have **large separation between class means** and **small spread inside each class**. This is Fisher's

criterion: $\max_w \frac{w^T S_B w}{w^T S_W w}$ where S_B measures between-class variability and S_W measures within-class variability. Once w has been learned, classification is performed by projecting a test sample x_t : $s_t = w^T x_t$. The scalar value s_t is a **classification score**. Then we compare it with a threshold t :

$$C(x_t) = \begin{cases} C_1 & \text{if } w^T x_t < t \\ C_2 & \text{if } w^T x_t \geq t \end{cases}$$

So the LDA classifier is linear: the decision boundary in the original space is a hyperplane orthogonal to w . The orientation of w matters. If we choose:

$$w = \alpha S_W^{-1} (\mu_2 - \mu_1), \alpha > 0$$

then the projected mean of class C_2 is larger than the projected mean of class C_1 . This makes the threshold rule consistent:

$$\begin{aligned} w^T x < t &\Rightarrow C_1 \\ w^T x \geq t &\Rightarrow C_2 \end{aligned}$$

For multiclass problems, LDA is usually used as **dimensionality reduction**. Instead of finding one direction w , we find a matrix of directions: $W = [w_1, \dots, w_m]$. W is the projection matrix. $W \in \mathbb{R}^{n \times m}$

and project: $y = W^T x$. The columns of W are the most discriminant directions. Unlike PCA, these directions are not required to be orthogonal. After this projection, we train a classifier on the reduced features y . For example: $x \in \mathbb{R}^n$ becomes:

$$y = W^T x \in \mathbb{R}^m$$

Then y can be classified with a Gaussian classifier, logistic regression, SVM, nearest neighbor, or another classifier. A key limitation is that, with K classes, LDA can extract at most: $K - 1$ useful discriminant directions. For example, the slides mention that with MNIST and 10 classes, LDA gives at most 9 directions.

In practice, when the feature dimension is large, S_W may be singular or almost singular. The slides therefore suggest applying PCA first and then LDA:

$x \rightarrow \text{PCA features} \rightarrow \text{LDA features} \rightarrow \text{classifier}$

This reduces dimensionality and makes the LDA computation more stable.

Objective function of multiclass LDA

Within-class covariance : $S_W = \sum_{c=1}^K \sum_{i:c_i=c} (x_i - \mu_c)(x_i - \mu_c)^T$

where:

- K = number of classes
- μ_c = mean of class c

A small S_W means that samples of the same class are compact.

This measures the distance between class means:

$$S_B = \sum_{c=1}^K N_c (\mu_c - \mu)(\mu_c - \mu)^T$$

where:

- N_c = number of samples of class c
- μ = global dataset mean

A large S_B means that class centroids are far apart.

LDA searches for projection directions that maximize between-class variability while minimizing within-class variability.

The multiclass objective function is:

$$W^* = \arg \max_W \frac{|W^T S_B W|}{|W^T S_W W|}$$

or equivalently: $\max_W \text{Tr} \left(\begin{pmatrix} S_B & 0 \\ 0 & S_W \end{pmatrix} \right)$

The solution is obtained from the generalized eigenvalue problem: $S_B W = \lambda S_W W$

The optimal columns of W are the eigenvectors associated with the largest generalized eigenvalues.

After projection: $y = W^T x$

classification is performed in the reduced space using another classifier

LIMITATIONS:

The main limitation is that LDA can produce at most: $K - 1$

useful projection directions, where K is the number of classes.

This happens because the rank of the between-class scatter matrix satisfies:

$$\text{rank}(S_B) \leq K - 1$$

Therefore:

- binary classification $\rightarrow$ at most 1 LDA direction
- 10 classes $\rightarrow$ at most 9 directions

Even if the original feature space is extremely high-dimensional, LDA cannot extract more than $K - 1$ discriminant components.

When the number of features is much larger than the number of training samples, the within-class scatter matrix: S_W may become **singular** or poorly conditioned.

A matrix becomes singular when some of its eigenvalues are equal to zero. This happens when there is no variance along some directions of the feature space. For example, if the data actually lie in a lower-dimensional subspace, some directions contain no information.

In high-dimensional problems (e.g. thousands of image pixels but only a few training samples), the data cannot span the whole feature space. As a consequence, some eigenvalues of S_W become zero, and the inverse:

$$S_W^{-1}$$

cannot be computed.

To solve this problem, PCA is often applied before LDA:

$$x \rightarrow \text{PCA} \rightarrow \text{LDA}$$

PCA removes directions with very small or zero variance and reduces dimensionality. This makes S_W smaller, more stable, and invertible, allowing LDA to work correctly.

MULTICLASS LOGISTIC REGRESSION

We have a classification problem with K classes: $C \in \{1, \dots, K\}$, given an input vector: $x \in \mathbb{R}^d$ we want to compute: $P(C = j \mid x)$ for every class j . Logistic regression is a **discriminative model**, meaning:

- we do NOT model $f_{X|C}(x \mid c)$
- we directly model posterior probabilities $P(C = j \mid x)$

We assume that each class has a linear score: $s_j(x) = w_j^T x + b_j$ where:

- $w_j \in \mathbb{R}^d$
- $b_j \in \mathbb{R}$

If the score $s_j(x)$ is large $\rightarrow$ class j is favored. A score cannot be a probability because it can be negative and it doesn't sum to 1, so we transform score into positive quantities using exponentials: $e^{s_j(x)} = e^{w_j^T x + b_j}$, so $P(C = j | x) \propto e^{w_j^T x + b_j}$

and this means: $P(C = j | x) = \frac{e^{w_j^T x + b_j}}{Z(x)}$ where $Z(x)$ is a normalization term. The sum of the probabilities must be = 1: $\sum_{j=1}^K P(C = j | x) = 1$, so $\sum_{j=1}^K \frac{e^{w_j^T x + b_j}}{Z(x)} = 1 \rightarrow \frac{1}{Z(x)} \sum_{j=1}^K e^{w_j^T x + b_j} = 1 \rightarrow Z(x) = \sum_{j=1}^K e^{w_j^T x + b_j}$.

We substitute $Z(x)$ and we obtain the SOFTMAX FUNCTION:

$$P(C = j | x) = \frac{e^{w_j^T x + b_j}}{\sum_{k=1}^K e^{w_k^T x + b_k}}$$

Take logarithm: $\log P(C = j | x) = \log \frac{e^{w_j^T x + b_j}}{\sum_{k=1}^K e^{w_k^T x + b_k}} \rightarrow \log e^{w_j^T x + b_j} -$

$\log \left(\sum_{k=1}^K e^{w_k^T x + b_k} \right)$. Since: $\log e^u = u$, we get: $= w_j^T x + b_j - \log \left(\sum_{k=1}^K e^{w_k^T x + b_k} \right)$

Define: $k(x) = -\log \left(\sum_{k=1}^K e^{w_k^T x + b_k} \right) \rightarrow \boxed{\log P(C = j | x) = w_j^T x + b_j + k(x)}$

$k(x)$ depends on x , but NOT on class j , so it can be trascurated.

So given the training dataset: Dataset: $D = \{(x_i, c_i)\}_{i=1}^n$, for sample x_i we have

$y_{ij} = P(C_i = j | x_i, W, b) = \frac{e^{w_j^T x_i + b_j}}{\sum_{k=1}^K e^{w_k^T x_i + b_k}}$. The vector $y_i = (y_{i1}, \dots, y_{iK})$ contains all

predicted probabilities.

Then we define a one-hot encoding of labels (only the correct class has value 1) :

$z_i = (z_{i1}, \dots, z_{iK})$ with:

$$z_{ij} = \begin{cases} 1 & \text{if } c_i = j \\ 0 & \text{otherwise} \end{cases}$$

and for one sample x_i we obtain the probability of observing the correct label:

$$P(c_i | x_i, W, b) = \prod_{j=1}^K y_{ij}^{z_{ij}}$$

If the samples of the dataset are i.i.d, we obtain the likelihood of the dataset:

$$L(W, b) = \prod_{i=1}^n \prod_{j=1}^K y_{ij}^{z_{ij}}$$

and the log_likelihood, using $\log \prod a_i = \sum \log a_i$:

$$\ell(W, b) = \log L(W, b) = \log \prod_{i=1}^n \prod_{j=1}^K y_{ij}^{z_{ij}} = \sum_{i=1}^n \sum_{j=1}^K \log (y_{ij}^{z_{ij}})$$

$$\rightarrow \boxed{\ell(W, b) = \sum_{i=1}^n \sum_{j=1}^K z_{ij} \log y_{ij}}$$

We estimate parameters by maximizing likelihood:

$$\rightarrow \boxed{(W^*, b^*) = \arg \max_{W, b} \ell(W, b)}$$

Define multiclass cross-entropy:

$$\rightarrow H(z_i, y_i) = - \sum_{j=1}^K z_{ij} \log y_{ij}$$

→ Therefore:

$$\rightarrow \boxed{J(W, b) = -\ell(W, b) = \sum_{i=1}^n H(z_i, y_i)}$$

→ So minimizing cross-entropy is equivalent to maximizing likelihood.

Substitute softmax:

$$\rightarrow J(W, b) = - \sum_{i=1}^n \sum_{j=1}^K z_{ij} \log \frac{e^{w_j^T x_i + b_j}}{\sum_{k=1}^K e^{w_k^T x_i + b_k}}$$

→ Only the correct class survives:

$$\rightarrow = - \sum_{i=1}^n \log \frac{e^{w_{c_i}^T x_i + b_{c_i}}}{\sum_{k=1}^K e^{w_k^T x_i + b_k}}$$

Finally:

$$J(W, b) = \sum_{i=1}^n \left[\log \sum_{k=1}^K e^{w_k^T x_i + b_k} - w_{c_i}^T x_i - b_{c_i} \right]$$

This is the SOFTMAX LOSS.

To avoid overfitting, add regularization: $R(W, b) = \Omega(W) + \frac{1}{n} J(W, b)$

Usually: $\Omega(W) = \frac{\lambda}{2} \sum_{k=1}^K \|w_k\|^2$

Thus:

$$R(W, b) = \frac{\lambda}{2} \sum_{k=1}^K \|w_k\|^2 + \frac{1}{n} \sum_{i=1}^n \left[\log \sum_{k=1}^K e^{w_k^T x_i + b_k} - w_{c_i}^T x_i - b_{c_i} \right]$$

17. Final classification rule

For a new sample x :

Compute all scores: $s_k(x) = w_k^T x + b_k$

Choose the class with maximum posterior probability: $\hat{c} = \arg \max_k P(C = k | x)$

Since softmax preserves ordering:

$$\hat{c} = \arg \max_k (w_k^T x + b_k)$$

MULTIVARIATE GAUSSIAN CLASSIFIER

We start from a **closed-set classification problem**: given a sample x_t , we want to assign it to one class $c \in \{1, \dots, K\}$. The Optimal Bayes decision assigns the class with maximum posterior probability:

$$c_t^* = \arg \max_c P(C_t = c | X_t = x_t)$$

The sample is modeled as a realization of a random variable: X_t and the class label is modeled as a random variable: C_t . We introduce a generative classifier which models the joint distribution of X and C : $f_{X,C}(x, c)$. It describes the probability density of

observing both the feature vector x and the class label c together.

We assume that the samples are independent and they follow the same distribution, according to $X_t, C_t \sim X, C$, for any test sample t , so : $f_{X_t, C_t}(x_t, c) = f_{X, C}(x_t, c)$.

Using Bayes rule, the posterior probability can be written as:

$$P(C_t = c | X_t = x_t) = \frac{f_{X, C}(x_t, c)}{\sum_{c'} f_{X, C}(x_t, c')}$$

The denominator is the marginal density of the observed sample:

$$f_X(x_t) = \sum_{c'} f_{X, C}(x_t, c')$$

So the posterior is:

$$P(C_t = c | X_t = x_t) = \frac{f_{X, C}(x_t, c)}{f_X(x_t)}$$

If we know the joint density, we can compute the posterior.

We can factorize the joint density as: $f_{X, C}(x, c) = f_{X|C}(x | c)P(C = c)$.

$f_{X|C}(x | c) \rightarrow$ this term is the **CLASS-CONDITIONAL DENSITY**. It describes the distribution of the samples of each class.

$\pi_c = P(C = c) \rightarrow$ is the **CLASS PRIOR PROBABILITY**. This describes how likely class c is before observing the sample. This is the application prior, it doesn't depend on the model parameters. It is a task-specific knowledge. While the **EMPIRICAL PRIOR** is the frequency of class c in a dataset: $\pi_c^{emp} = \frac{N_c}{N}$. In general: $\pi_c^{emp} \neq \pi_c$.

Therefore:

$$P(C = c | X = x) = \frac{f_{X|C}(x | c)P(C = c)}{\sum_{c'} f_{X|C}(x | c')P(C = c')}$$

so the posterior is proportional to:

$$P(C = c | X = x) \propto f_{X|C}(x | c)P(C = c)$$

How do we model $f_{X|C}(x | c)$?

we assume that the data of each class can be modeled by a Multivariate Gaussian distribution. Therefore: $(X_t | C_t = c) \sim (X | C = c) \sim \mathcal{N}(\mu_c, \Sigma_c)$.

$$\theta = \{\mu_c, \Sigma_c, \pi_c\}_{c=1}^K$$

To learn these model parameters:

We start from our labelled training dataset: $D = \{(x_1, c_1), \dots, (x_N, c_N)\}$

We assume iid: assume that observations are independent and identically distributed given the model parameters: $[(X_i, C_i) \perp\!\!\!\perp (X_j, C_j)] \mid \theta$

and: $(X_i, C_i) \mid \theta \sim (X, C) \mid \theta$. So we are assuming that all the samples, both training and evaluation are independent and are distributed in the same way

We are following a FREQUENTIST approach→

We follow a frequentist approach, and we thus want to compute an **estimator** (or point estimate) θ^* of the model parameters

We will then use the estimated parameters to compute

$$f_{X_i|C_i}(x_i|c) \approx \mathcal{N}(x_i|\mu_c^*, \Sigma_c^*)$$

A possible way to estimate the model parameters is to MAXIMIZE the data LOG-LIKELIHOOD:

. For the assumption iid, we can factorize the likelihood as a product over samples:

$$L(\theta) = f(D \mid \theta) = \prod_{i=1}^N f_{X,C|\theta}(x_i, c_i \mid \theta) \text{ Using the generative decomposition:}$$

$$f_{X,C|\theta}(x_i, c_i \mid \theta) = f_{X|C,\theta}(x_i \mid c_i, \theta)P(c_i)$$

$$\text{Since: } f_{X|C,\theta}(x_i \mid c_i, \theta) = \mathcal{N}(x_i \mid \mu_{c_i}, \Sigma_{c_i})$$

$$\text{we obtain: } L(\theta) = \prod_{i=1}^N \mathcal{N}(x_i \mid \mu_{c_i}, \Sigma_{c_i})P(c_i)$$

We write the log-likelihood: $\ell(\theta) = \log L(\theta)$

$$\text{thus: } \ell(\theta) = \sum_{i=1}^N \log \mathcal{N}(x_i \mid \mu_{c_i}, \Sigma_{c_i}) + \sum_{i=1}^N \log P(c_i)$$

The second term does not depend on Gaussian parameters, therefore it can be ignored when maximizing w.r.t. μ_c, Σ_c .

$$\text{So the objective separates by class: } \ell(\theta) = \sum_{c=1}^K \ell_c(\mu_c, \Sigma_c) + \xi$$

$$\text{where: } \ell_c(\mu_c, \Sigma_c) = \sum_{i:c_i=c} \log \mathcal{N}(x_i \mid \mu_c, \Sigma_c)$$

and ξ collects terms independent of μ_c, Σ_c .

At this point, substituting the Gaussian density:

$$\mathcal{N}(x \mid \mu_c, \Sigma_c) = \frac{1}{(2\pi)^{d/2} |\Sigma_c|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c) \right)$$

we obtain:

$$\ell_c(\mu_c, \Sigma_c) = -\frac{N_c}{2} \log |\Sigma_c| - \frac{1}{2} \sum_{i:c_i=c} (x_i - \mu_c)^T \Sigma_c^{-1} (x_i - \mu_c) + \text{const}$$

So the likelihood depends on three quantities:

$$N_c, \sum_{i:c_i=c} x_i, \sum_{i:c_i=c} x_i x_i^T$$

Then, by setting derivatives to zero, the optimal solutions become:

$$\mu_c^* = \frac{1}{N_c} \sum_{i:c_i=c} x_i$$

$$\Sigma_c^* = \frac{1}{N_c} \sum_{i:c_i=c} (x_i - \mu_c^*)(x_i - \mu_c^*)^T$$

The covariance measures how samples of class c spread around their mean.

Then we can compute the likelihood of class c for a test point x_t as:

$$f_{X|C}(x_t | c) = \mathcal{N}(x_t | \mu_c^*, \Sigma_c^*)$$

Then we combine it with the prior: $f_{X|C}(x_t | c)P(C = c)$

$$\text{The posterior is: } P(C = c | X = x_t) = \frac{\mathcal{N}(x_t | \mu_c^*, \Sigma_c^*)P(C=c)}{\sum_{c'} \mathcal{N}(x_t | \mu_{c'}^*, \Sigma_{c'}^*)P(C=c')}$$

For classification, the denominator is the same for all classes, so it is not necessary to compute it if we only want the winning class.

$$\text{Therefore: } c_t^* = \arg \max_c \mathcal{N}(x_t | \mu_c^*, \Sigma_c^*)P(C = c)$$

In practice, we work in log-domain:

$$S_c(x_t) = \log \mathcal{N}(x_t | \mu_c^*, \Sigma_c^*) + \log P(C = c)$$

$$\text{and: } c_t^* = \arg \max_c S_c(x_t)$$